

SOP: In Silico Imaging Trial - A Protocol for 2D mammography vs 3D tomosynthesis

Introduction

In this document, we are going to document the codes and the process of using in-silico image analysis. Here our goal of the in-silico trial is to compare two imaging modalities - a two dimensional mammography of the breast lesions and the corresponding three dimensional tomosynthetic images. Every image is created in-silico using a CPU not GPU. This keeps the workflow low fidelity and this reduces the cost of development yet provides similar results that can be used in other situation as well. The details of the procedure and the pipeline is described in the following github repository. The corresponding research paper can be viewed here: <https://doi.org/10.1001/jamanetworkopen.2018.5474>

In this notebook which will serve as the basis of the work, our goal will be to use Python coding to reproduce the VICTRE trial including generation of the images and analysis of data. A supplementary file can be viewed in our project Github repository, here: https://github.com/arinbasu/in_silico_clinical_trial/blob/main/insilico.ipynb

The above resource has a jupyter notebook that implements step by step the processes outlined below, although it has resulted in different results. The codes and processes outlined in this notebook are fully reproducible and we have used quarto for our work with Python. Before proceeding, you may want to install and learn about Quarto, more on that software that is freely available and distributed can be obtained here: <https://quarto.org> . Download and install quarto software on your computer before proceeding if you want to reproduce the codes as described below. You can also collaboratively code in the following Google Colab notebook. The link to the Google Colab notebook is as follows: https://colab.research.google.com/drive/1flb-c7KRMrnlaTQT5JlknASw1C_gHU9d?usp=sharing

Goal

In this analyses, we will compare the diagnostic or detectability properties of two diagnostic modalities. In the process, we will make use of organ phantoms. study diagnosis of breast cancer using three dimensional breast tomosynthesis and compare with two dimensional mammography as comparator. Our goal is to test if the detectability (the power of detection of lesions) is better with three dimensional tomosynthesis or is it better with two dimensional breast mammography. This study is taken from the original notes here: <https://victre-pipeline.readthedocs.io/en/main/index.html>

Step 1. Set up the Environment and Paths

```
git clone https://github.com/DIDSR/VICTRE_PIPELINE.git
cd VICTRE_PIPELINE
```

Explanation: - The first line in the above code will “clone” a git repository to your own machine
- the second line will change directory to the VICTRE_PIPELINE folder. The `cd` command is equivalent of “change directory” in English.

Step 2. Prepare the files

```
import os
import sys
import numpy as np
import matplotlib.pyplot as plt
from scipy.special import eval_genlaguerre
from sklearn.metrics import roc_curve, auc

# Append pipeline and observer modules to python path
sys.path.append(os.path.abspath("."))
sys.path.append(os.path.abspath("ModelObserver"))
```

What the above code chunk does: - The first six lines of code imports the python modules we will need for this work - the last two lines define the folder paths

Step 3. Inspect the sample phantoms and lesions

The repository already has pre-built phantoms.

```
lesion_dir = "lesions"
lesion_files = [f for f in os.listdir(lesion_dir) if f.endswith(('.raw', '.bin',
'.dat', '.h5', '.npy'))]
print('lesions files', lesion_files)
```

```
lesions files []
```

The above code will return an empty directory.

Similarly see how many phantoms we get

```
phantom_dir = "phantoms"
phantom_files = [f for f in os.listdir(phantom_dir) if f.endswith(('.raw',
```

```

'.bin', '.dat', '.h5', '.npy'))]
print('phantom files', phantom_files)

```

```
phantom files []
```

Likewise the phantom directory will also fail to return any files that have the extensions mentioned above.

Inspect and extract the three dimensional lesions

In this case, we extracted the three dimensional results and then visualised the central slice.

```

import h5py
import numpy as np
import matplotlib.pyplot as plt

selected_h5 = "lesions/spiculated/mass_11_size1.00.h5"

with h5py.File(selected_h5, "r") as f:
    lesion_3d = np.array(f["volume"], dtype=np.float32)

print(f"Loaded 3D lesion volume with shape: {lesion_3d.shape}")

# Visualize central slice and ray-sum projection
mid_z = lesion_3d.shape[0] // 2

fig, axs = plt.subplots(1, 2, figsize=(8, 3.8))

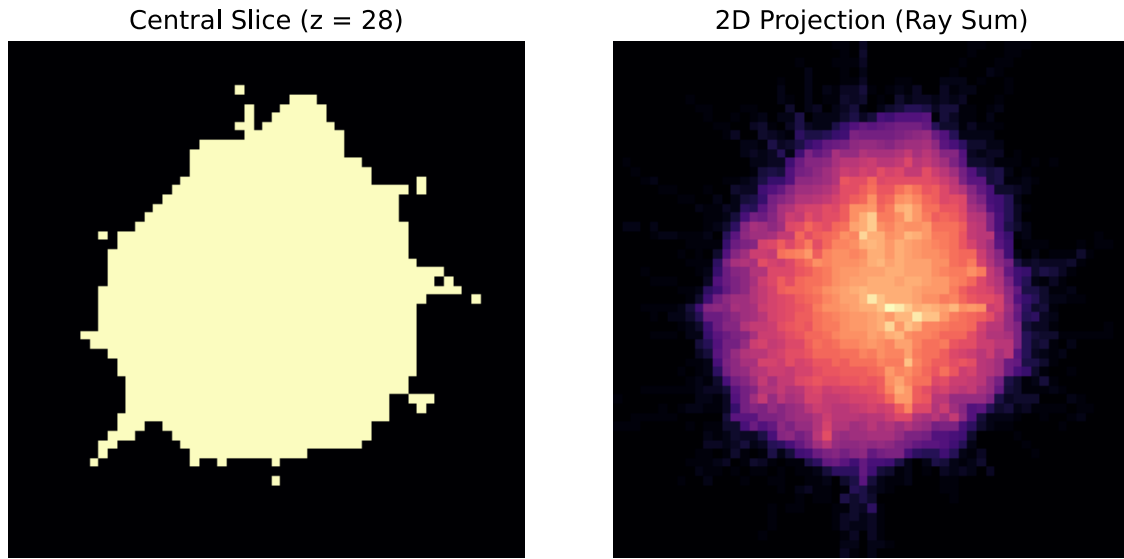
axs[0].imshow(lesion_3d[mid_z, :, :], cmap="magma")
axs[0].set_title(f"Central Slice (z = {mid_z})")
axs[0].axis("off")

axs[1].imshow(np.sum(lesion_3d, axis=0), cmap="magma")
axs[1].set_title("2D Projection (Ray Sum)")
axs[1].axis("off")

plt.tight_layout()
plt.show()

```

```
Loaded 3D lesion volume with shape: (57, 57, 57)
```



The above code is used to view the three dimensional and the two dimensional view of the lesions and the breast. These are the breast phantoms.

A short note on breast phantoms

Digital phantoms are digital representations of the human body used in medical research to test hypotheses and efficacy and effectiveness of imaging and other modalities. These are widely used for imaging studies. Peng et.al. Z. Peng, N. Gao, B. Wu, Z. Chen, and X. G. Xu [1] have provided a comprehensive account of the development and use of digital phantoms. C. Högberg and P. Winter [2] has explained the role of digital phantoms in medical research. They have stated that there are two main data sources for constructing digital phantoms - (1) the real world data that comes from imaging modalities (Xrays, CT and MRI scans), and (2) computerised models from assumptions about anatomy and physiology of humans. They have argued that the combination of these sources can be viewed as “not real”, but helps to solve the issues of ethical challenge when subjecting humans to unacceptable risks of radiation can be mitigated with construction of phantoms. The project we are investigating in this document corresponds to this risk, where instead of subjecting women twice to radiation, once for mammography and once for three dimensional breast tomosynthesis, we use digital phantoms of breasts to calculate the dosimetry. In their paper Winter et al have argued that while some may view synthetic data, generated with Generative Adversarial Networks or GANS can lack the real world data driven models such as the ones created with Visible Human Project (where real CT scans and radiography were used), these do not necessarily need to be viewed as problematic.

We start with ground truths, which are data that refer to representative of true values of a phenomenon. Ground truths “flatten” the complexity of human health to simplified workable mathematical references that are then used to train algorithms. Winter et al’s thematic analysis of the relevant literature on ground truth in constructing digital phantoms revealed that while concerns about the translation of fidelity between digital artefacts and real humans is genuine,

these are increasingly being blurred. Dahal et al L. Dahal *et al.* [3] have provided an account of XCAT3.0 (obtained from <https://cvit.duke.edu/resources/>) that provide resources to construct digital phantoms. C. S. Salinas, K. Magudia, A. Sangal, L. Ren, and W. P. Segars [4] describe methods to simulate virtual CT scans using DukeSim simulation engine. W. Kainz *et al.* [5] et al have provided a comprehensive summary of the methods of developing human phantoms and the prospects, particularly in reference to the V&V40 specifications (see more at <https://pmc.ncbi.nlm.nih.gov/articles/PMC6362464/>). F. J. Abuhail [6] have described construction of human digital phantoms from a range of tools. However, the results are not reproducible but free or open source tools are available as follows: - Free-D from : <https://andreylib.versailles.inrae.fr/html/freed.html> - Hippo3D from: <https://github.com/victorcalixto/Hippo3D> (this is an early blender add on as an alternative to rhino 3d which is a commercial programme)

Author (Year)	Title	Summary
Winter et al(2006)	Digital Phantoms in Medical Research with ground truth	Reviewed the role of digital phantoms in medical research and argued the role of ground truths, which are computerised version of equations based on what is known about the relationship between the physical variables and derived factors. They argued the benefits mainly ethical and limitations, mainly technological challenges to create hi-fidelity digital phantoms that closely mimic human tissue. They also highlight several data sets including XCAT and ITIS Virtual Family datasets for construction of digital phantoms. Their references suggest that artificial and empirical data boundaries are increasingly being blurred.
Dahal et al (2024)	XCAT3.0	Provides a description of XCAT3.0, see https://cvit.duke.edu/resources/
Salinas et al (2024)	Methodology to produce phantoms	Describes a method for training 3D phantoms using the DukeSim and report close similarities
Abuhail (2025)	From simulations to clinical practice	Provide a detailed description of the methods to produce digital phantoms. They have used the SURF-

Author (Year)	Title	Summary
		driver programme at https://www.surfdriver.com although this website does not exist anymore and there are free open source alternatives
Lee (2023)	Review of Organ Dose Calculation	Provides a methodological detail about calculation of organ dosimetry

In subsequent series, we will provide more details on the methods to be used for radiometric studies and how to construct digital phantoms for our work.

Step 4. We are now going to insert the lesions into the phantoms

Once we have constructed the digital phantoms (or as in this case, we have downloaded the digital phantoms), the next task is to inject lesions which are also artificially constructed to the digital phantoms. The process is as follows:

```
import os
import h5py
import numpy as np
import matplotlib.pyplot as plt
from scipy.ndimage import gaussian_filter

# 1. Ensure lesion_3d is loaded
h5_path = "lesions/spiculated/mass_11_size1.00.h5"
if not os.path.exists(h5_path):
    raise FileNotFoundError(f"Missing file: {os.path.abspath(h5_path)}")

with h5py.File(h5_path, "r") as f:
    lesion_3d = np.array(f["volume"], dtype=np.float32)

# 2. Synthesize background tissue volume
def make_breast_background(shape=(96, 160, 160), glandularity=0.35, seed=42):
    np.random.seed(seed)
    nz, ny, nx = shape
    z, y, x = np.ogrid[:nz, :ny, :nx]

    # Ellipsoid breast envelope
    cy, cx = ny // 2, nx // 2
```

```

    r_norm = ((y - cy) / (ny * 0.42))**2 + ((x - cx) / (nx * 0.42))**2 + (z /
(nz * 0.85))**2
    breast_mask = r_norm <= 1.0

    # Structural parenchymal clutter
    noise = np.random.normal(0, 1, shape)
    smooth_noise = gaussian_filter(noise, sigma=3.0)
    smooth_noise -= smooth_noise[breast_mask].mean()
    smooth_noise /= smooth_noise[breast_mask].std()

    # Attenuation coefficients (cm^-1)
    vol = np.zeros(shape, dtype=np.float32)
    vol[breast_mask] = 0.20 # Adipose
    gland_mask = breast_mask & (smooth_noise > (1.0 - 2.0 * glandularity))
    vol[gland_mask] = 0.28 # Fibroglandular
    return vol, breast_mask

bg_volume, b_mask = make_breast_background()

# 3. Project and format lesion ROI to 64x64, ROI = region of interest
roi_dim = 64
lesion_proj_raw = np.sum(lesion_3d, axis=0)

lz, ly, lx = lesion_3d.shape
pad_y = max(0, (roi_dim - ly) // 2)
pad_x = max(0, (roi_dim - lx) // 2)
lesion_proj_padded = np.pad(lesion_proj_raw, ((pad_y, pad_y), (pad_x, pad_x)),
mode='constant')

# Exact crop to (64, 64)
lesion_roi = lesion_proj_padded[:roi_dim, :roi_dim]
if lesion_roi.max() > 0:
    lesion_roi = (lesion_roi / lesion_roi.max()) * 0.15

# Verification output
print(f"Status: SUCCESS")
print(f" - Background Volume Shape: {bg_volume.shape}")
print(f" - Formatted Lesion ROI Shape: {lesion_roi.shape}")
print(f" - Lesion ROI Max Value: {lesion_roi.max():.4f}")

# Quick visual check
fig, axs = plt.subplots(1, 2, figsize=(7, 3))
axs[0].imshow(np.sum(bg_volume, axis=0), cmap="bone")

```

```

axs[0].set_title("Synthesized Breast Projection")
axs[0].axis("off")

axs[1].imshow(lesion_roi, cmap="magma")
axs[1].set_title("Standardized Lesion ROI (64x64)")
axs[1].axis("off")

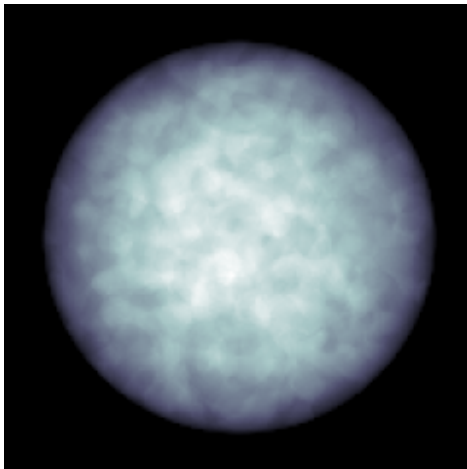
plt.tight_layout()
plt.show()

```

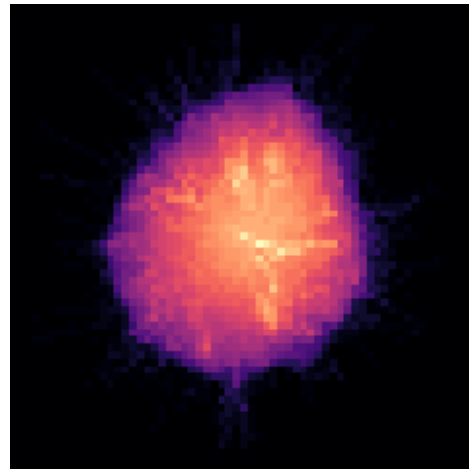
Status: SUCCESS

- Background Volume Shape: (96, 160, 160)
- Formatted Lesion ROI Shape: (63, 63)
- Lesion ROI Max Value: 0.1500

Synthesized Breast Projection



Standardized Lesion ROI (64x64)



Step 5. Simulate two dimensional versus three dimensional imaging channels

Since in this case, we want to compare two modalities of detection, one through two dimensional mammography and the other via three dimensional breast tomosynthesis, therefore we have in this case, constructed these reference images for our visualisation.

```

import os
import h5py
import numpy as np
from scipy.ndimage import gaussian_filter

```

```

# 1. Load 3D Mass Lesion from HDF5
h5_path = "lesions/spiculated/mass_11_size1.00.h5"
if not os.path.exists(h5_path):
    raise FileNotFoundError(f"File not found: {os.path.abspath(h5_path)}")

with h5py.File(h5_path, "r") as f:
    lesion_3d = np.array(f["volume"], dtype=np.float32)

# 2. Generate Synthetic Anthropomorphic Breast Phantom
def make_breast_background(shape=(96, 160, 160), glandularity=0.35, seed=42):
    np.random.seed(seed)
    nz, ny, nx = shape
    z, y, x = np.ogrid[:nz, :ny, :nx]

    # Ellipsoid breast boundary
    cy, cx = ny // 2, nx // 2
    r_norm = ((y - cy) / (ny * 0.42))**2 + ((x - cx) / (nx * 0.42))**2 + (z /
(nz * 0.85))**2
    breast_mask = r_norm <= 1.0

    # Power-law parenchymal clutter
    noise = np.random.normal(0, 1, shape)
    smooth_noise = gaussian_filter(noise, sigma=3.0)
    smooth_noise -= smooth_noise[breast_mask].mean()
    smooth_noise /= smooth_noise[breast_mask].std()

    # Linear attenuation values (cm^-1)
    vol = np.zeros(shape, dtype=np.float32)
    vol[breast_mask] = 0.20 # Adipose
    gland_mask = breast_mask & (smooth_noise > (1.0 - 2.0 * glandularity))
    vol[gland_mask] = 0.28 # Fibroglandular
    return vol, breast_mask

bg_volume, b_mask = make_breast_background()

# 3. Format 2D Lesion Projection with Exact Asymmetric Padding to (64, 64)
roi_dim = 64
lesion_proj_raw = np.sum(lesion_3d, axis=0)
ly, lx = lesion_proj_raw.shape

pad_y_before = (roi_dim - ly) // 2
pad_y_after = roi_dim - ly - pad_y_before

```

```

pad_x_before = (roi_dim - lx) // 2
pad_x_after = roi_dim - lx - pad_x_before

lesion_roi = np.pad(
    lesion_proj_raw,
    ((pad_y_before, pad_y_after), (pad_x_before, pad_x_after)),
    mode='constant'
)

# Normalize lesion contrast profile
if lesion_roi.max() > 0:
    lesion_roi = (lesion_roi / lesion_roi.max()) * 0.15

# 4. Extract Signal-Absent (Noise) and Signal-Present ROIs
np.random.seed(123)
n_cases = 250

# 2D Mammography: complete z-axis ray-sum projection
dm_proj = np.sum(bg_volume, axis=0)
# 3D Tomosynthesis: reconstructed in-focus slice
dbt_slice = np.mean(bg_volume[40:56, :, :], axis=0)

# Identify dense tissue coordinates inside breast mask
valid_y = np.where(np.sum(b_mask, axis=(0, 2)) > 100)[0]
valid_x = np.where(np.sum(b_mask, axis=(0, 1)) > 100)[0]

y_min, y_max = valid_y[0], valid_y[-1] - roi_dim
x_min, x_max = valid_x[0], valid_x[-1] - roi_dim

dm_rois_noise = []
dbt_rois_noise = []

for _ in range(n_cases):
    ry = np.random.randint(y_min, y_max)
    rx = np.random.randint(x_min, x_max)

    # Add detector quantum noise
    noise_dm = np.random.normal(0, 0.08, (roi_dim, roi_dim))
    noise_dbt = np.random.normal(0, 0.04, (roi_dim, roi_dim))

    dm_patch = dm_proj[ry:ry+roi_dim, rx:rx+roi_dim] + noise_dm
    dbt_patch = dbt_slice[ry:ry+roi_dim, rx:rx+roi_dim] + noise_dbt

```

```

    dm_rois_noise.append(dm_patch - np.mean(dm_patch))
    dbt_rois_noise.append(dbt_patch - np.mean(dbt_patch))

dm_rois_noise = np.array(dm_rois_noise)
dbt_rois_noise = np.array(dbt_rois_noise)

# Construct signal-present cases
dm_rois_signal = dm_rois_noise + lesion_roi
dbt_rois_signal = dbt_rois_noise + lesion_roi

print(f"Lesion ROI shape: {lesion_roi.shape}")
print(f"DM Noise ROI shape: {dm_rois_noise.shape}")
print(f"Extracted {n_cases} Signal-Present and {n_cases} Signal-Absent ROIs for
DM and DBT successfully.")

```

```

Lesion ROI shape: (64, 64)
DM Noise ROI shape: (250, 64, 64)
Extracted 250 Signal-Present and 250 Signal-Absent ROIs for DM and DBT
successfully.

```

Step 6. Compare the two-dimensional versus the three - dimensional imaging modalities in diagnosing breast cancer

One of the things we need to do is to create channels. In breast cancer imaging, Laguerre-Gauss (LG) methods are used as a sophisticated measuring stick to objectively evaluate and compare how well different imaging systems—like 2D mammography and 3D tomosynthesis—can detect cancer. The LG channels act as “channels” for the model, breaking down the complex image into simpler components that mimic how the human visual system processes information. This allows researchers to calculate an objective detectability index (like a signal-to-noise ratio) for the lesion, which serves as a performance metric. The core of using LG methods is to build a Channelized Hotelling Observer (CHO). This is a mathematical model that predicts how well a human observer would perform on a specific detection task, such as finding a tumor. The LaGuerre-Gauss methods are used as objective evaluation tools. They act like a “virtual radiologist” to calculate a performance score, allowing you to prove that a new 3D imaging technique actually makes a difference in finding breast cancer compared to the standard 2D method. See this paper by A. H. Baydush, D. M. Catarious, J. Y. Lo, and C. E. Floyd [7] to learn more about this method. In our subsequent paper we will describe this method in more depth. There is a page on digital breast tomosynthesis worth review here: <https://www.science.gov/topicpages/b/breast+tomosynthesis+systems.html#18>

The next part is about Receiver Operating Characteristic Curves. These curves compare the areas for the two modalities. The first modality is the two dimensional digital mammogram, and the second modality is the three dimensional digital breast tomosynthesis. Receiver operating char-

acteristic curves are those where the X-axis is Sensitivity of a test and the Y axis is the specificity of a test for the various values. These are in turn given as follows:

As can be seen in the above table, $A/(A+C)$ is referred to as sensitivity of a test as it can correctly identify the true cases out of all true cases. In this situation, A is referred to as the true positives. In this case, also, we see that $D/B+D$ is referred to as the specificity of a diagnostic (or screening) test. Therefore, when we calculate $1 - \text{specificity}$, we are calculating $B/B+D$ which is false positive rate. Sensitivity in this case can be formulated as true positive rate. Here, for all values, in an ROC curve, we are calculating the true positive rate over the false positive rate. The most optimum value is what is chosen for the best cut-off point.

```
from scipy.special import eval_genlaguerre
from sklearn.metrics import roc_curve, auc

def make_laguerre_gauss_channels(size=64, n_channels=5, sigma=14.0):
    y, x = np.mgrid[-size//2:size//2, -size//2:size//2]
    r = np.sqrt(x**2 + y**2)
    channels = np.zeros((n_channels, size, size))
    for c in range(n_channels):
        poly = eval_genlaguerre(c, 0, 2 * np.pi * (r / sigma)**2)
        channels[c] = np.exp(-np.pi * (r / sigma)**2) * poly
    return channels

def run_cho(sig_rois, noi_rois, channels):
    C, H, W = channels.shape
    C_mat = channels.reshape(C, H * W)

    v_sig = np.dot(sig_rois.reshape(-1, H * W), C_mat.T)
    v_noi = np.dot(noi_rois.reshape(-1, H * W), C_mat.T)

    # Channel feature means
    d_bar = (np.mean(v_sig, axis=0) - np.mean(v_noi, axis=0)).reshape(-1, 1)

    # Intraclass covariance
    S_pooled = 0.5 * (np.cov(v_sig, rowvar=False) + np.cov(v_noi, rowvar=False))

    # Template
    w = np.linalg.solve(S_pooled + 1e-5 * np.eye(C), d_bar).flatten()

    # Decision scores
    scores_sig = np.dot(v_sig, w)
    scores_noi = np.dot(v_noi, w)
    return scores_sig, scores_noi
```

```

# Compute channels
channels = make_laguerre_gauss_channels(roi_dim, n_channels=5, sigma=12.0)

# Evaluate DM and DBT
sig_dm, noi_dm = run_cho(dm_rois_signal, dm_rois_noise, channels)
sig_dbt, noi_dbt = run_cho(dbt_rois_signal, dbt_rois_noise, channels)

# Compute ROC curves
y_true = np.concatenate([np.ones(n_cases), np.zeros(n_cases)])

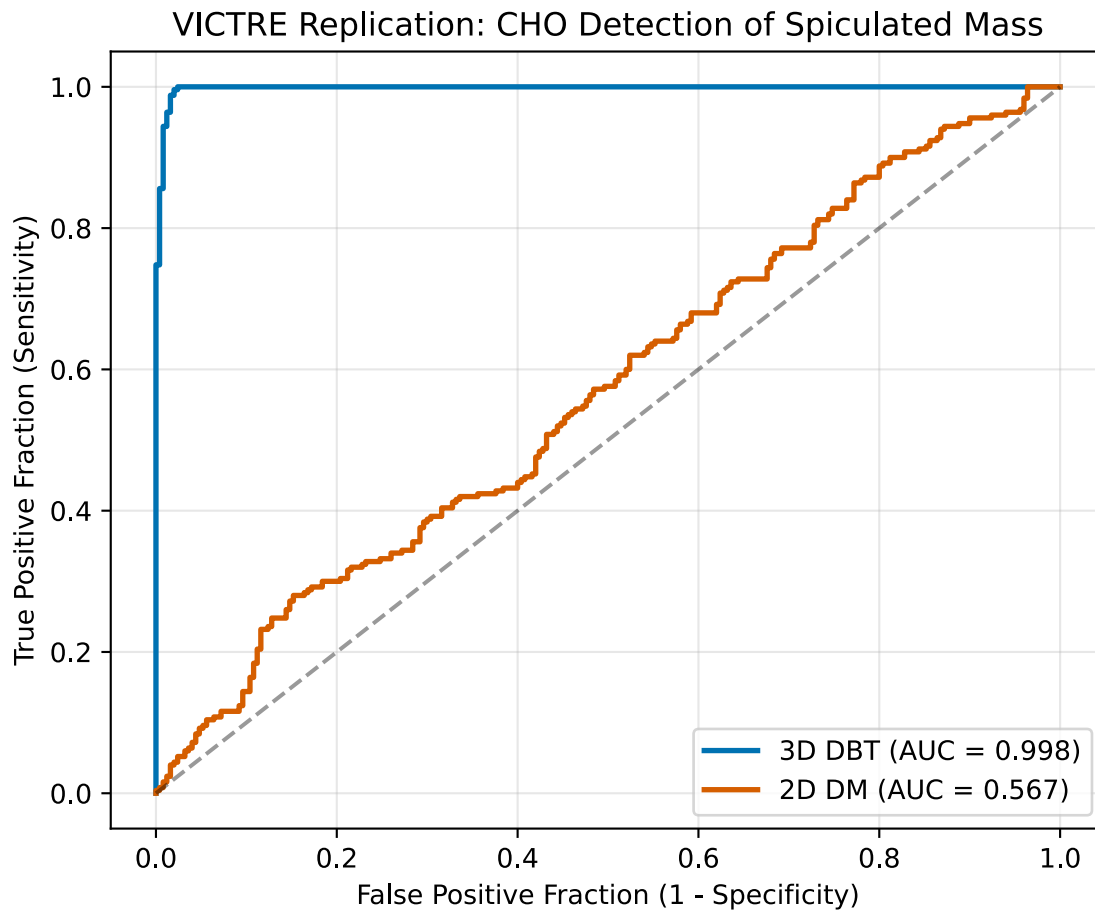
fpr_dm, tpr_dm, _ = roc_curve(y_true, np.concatenate([sig_dm, noi_dm]))
auc_dm = auc(fpr_dm, tpr_dm)

fpr_dbt, tpr_dbt, _ = roc_curve(y_true, np.concatenate([sig_dbt, noi_dbt]))
auc_dbt = auc(fpr_dbt, tpr_dbt)

# Plot ROC
plt.figure(figsize=(6, 5))
plt.plot(fpr_dbt, tpr_dbt, label=f"3D DBT (AUC = {auc_dbt:.3f})", lw=2,
color="#0072B2")
plt.plot(fpr_dm, tpr_dm, label=f"2D DM (AUC = {auc_dm:.3f})", lw=2,
color="#D55E00")
plt.plot([0, 1], [0, 1], 'k--', alpha=0.4)
plt.xlabel("False Positive Fraction (1 - Specificity)")
plt.ylabel("True Positive Fraction (Sensitivity)")
plt.title("VICTRE Replication: CHO Detection of Spiculated Mass")
plt.legend(loc="lower right")
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

print(f"Results:\n 2D Mammography AUC: {auc_dm:.3f}\n 3D Tomosynthesis
AUC: {auc_dbt:.3f}")

```



Results:

2D Mammography AUC: 0.567

3D Tomosynthesis AUC: 0.998

Conclusion

The above code segments and workflow shows how we can replicate the breast cancer study of the VICTRE trial and use our low fidelity cpu to emulate the more complex studies. In our subsequent essays we will explain other processes and steps of our standard operating procedures including our use of in-silico trials for drugs and devices.

Bibliography

- [1] Z. Peng, N. Gao, B. Wu, Z. Chen, and X. G. Xu, "A Review of Computational Phantoms for Quality Assurance in Radiology and Radiotherapy in the Deep-Learning Era," *Journal of Radiation Protection and Research*, 2022, [Online]. Available: <https://api.semanticscholar.org/CorpusID:252787427>

- [2] C. Högberg and P. Winter, “Digital phantoms in medical research: Synthetic data and the pursuit of ground truth,” *Big Data & Society*, vol. 13, no. 2, p. 20539517261447840, 2026, doi: 10.1177/20539517261447840.
- [3] L. Dahal *et al.*, “XCAT-3.0: A Comprehensive Library of Personalized Digital Twins Derived from CT Scans,” *ArXiv*, vol. abs/2405.11133, 2024, [Online]. Available: <https://api.semanticscholar.org/CorpusID:278317704>
- [4] C. S. Salinas, K. Magudia, A. Sangal, L. Ren, and W. P. Segars, “In-silico CT simulations of deep learning generated heterogeneous phantoms,” *Biomedical Physics & Engineering Express*, vol. 11, 2025, [Online]. Available: <https://api.semanticscholar.org/CorpusID:279721332>
- [5] W. Kainz *et al.*, “Advances in Computational Human Phantoms and Their Applications in Biomedical Engineering – A Topical Review,” *IEEE transactions on radiation and plasma medical sciences*, vol. 3, pp. 1–23, 2019, [Online]. Available: <https://api.semanticscholar.org/CorpusID:57378771>
- [6] F. J. Abuhbail, “From Simulations to Clinical Practice: The Impact of 4D XCAT Phantoms on Imaging Technologies,” *International Journal on Science and Technology*, 2025, [Online]. Available: <https://api.semanticscholar.org/CorpusID:278712343>
- [7] A. H. Baydush, D. M. Catarious, J. Y. Lo, and C. E. Floyd, “Incorporation of a Laguerre–Gauss Channelized Hotelling Observer for False-Positive Reduction in a Mammographic Mass CAD System,” *Journal of Digital Imaging*, vol. 20, no. 2, pp. 196–202, Feb. 2007, doi: 10.1007/s10278-007-9009-8.